Knowledge Graph Generator: Specification

Teamprojekt Softwareentwicklung 2026/27, TU Darmstadt · Team 080
Client: Ning Xia, Research Data Management Group, Chair of Fluid Systems, TU Darmstadt
Team: Surkhabjit Singh, Mahdi Rafiei, Mohammad Riad Loutfi, Arian Faruqui, Denis Ücyildiz, M Adel Zitouni
Version: 13.11.2026

1. Project Description

Vision

Our client is building an information retrieval system that collects mostly unstructured data from different sources and makes it searchable. Knowledge graphs store its relations in a structured, queryable way, but building them by hand takes time and expert knowledge and leads to inconsistencies. The Knowledge Graph Generator (KGG) manages and versions graphs built from raw data and a given ontology, and lets a human decide where the system cannot. In the long term, the client wants the KGG to run permanently in the institute's backend as its own project that they keep developing.

Current and Target State

The institute runs a search portal with a React frontend and a Django backend that uses, among others, the graph database Fuseki. Contrary to the call for projects, the client made clear that generating the graph is not the focus, because it is still mostly a research problem. Instead, the KGG receives ready-made triples, so that a generation step can be added later. Its goals are a REST API to add, read, edit and update triples, versioning of every change with Linked Data Event Streams (LDES), checks against the existing graph that avoid duplicate entities, and a review step in which a person decides unclear cases (human-in-the-loop). We agreed with the client that the KGG gets its own backend with a Python core, called by the institute's backend over a REST API, plus a simple frontend for the reviewer. We propose to build this backend as a Django app that the institute could also install directly. The client values software the institute can maintain and extend, and code the team fully understands.

Domain

A knowledge graph stores facts as triples of subject, predicate and object. RDF is a standard format for these triples, and an ontology defines the allowed classes and relations. LDES describes changes to an RDF graph as a stream of versions, so older states can be restored. Three roles work with the KGG: the client system, currently the institute's backend, sends triples over our REST API and gets the results back; the reviewer decides unclear cases in the frontend; the maintainers, developers at the institute, run, integrate and further develop the KGG. Other stakeholders are the client, who sets priorities and accepts requirements, and our course supervisor. Constraints are the integration into the institute's Django backend, a local database for now, open-source publication and a final handover to the institute's GitLab.

Architecture

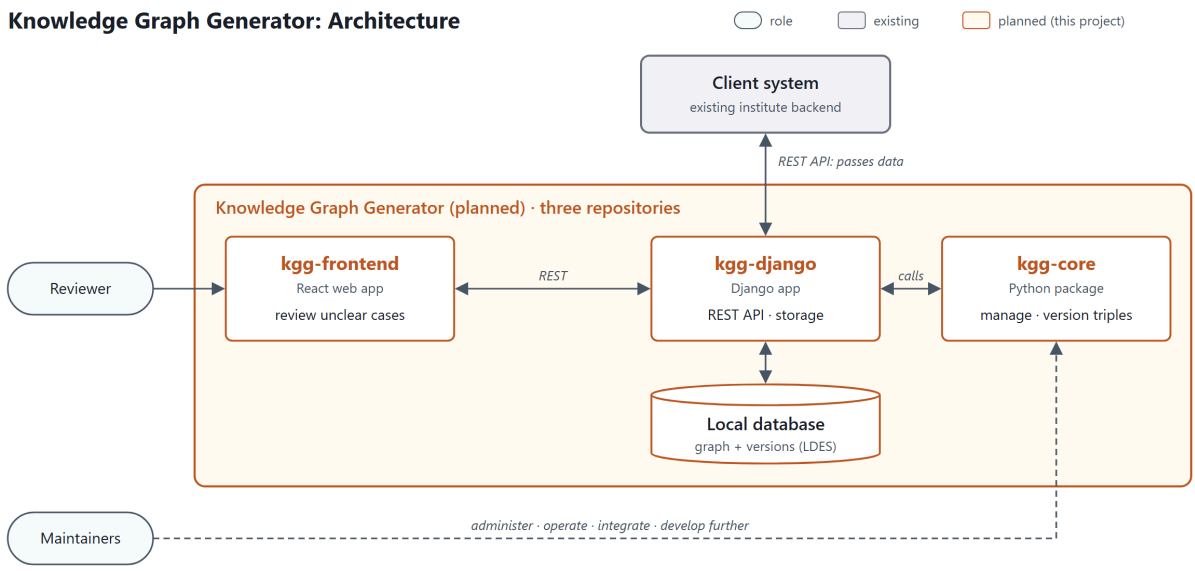


Figure 1: Planned architecture. The split into three repositories is our proposal and still has to be confirmed by the client.

2. Deliverables

User interface. The main interface of the Knowledge Graph Generator (KGG) is a REST API, through which the client's backend sends data and gets results. The REST API is part of a Django app (kgg-django) that the institute can also install directly in its own backend. The graph logic is a Python package without Django (kgg-core). For human-in-the-loop, a reviewer uses a simple React web application (kgg-frontend).

Development environment and tools. We develop in Python with Django and in React. Requirements are GitHub issues, iterations are planned in GitHub Projects, and every change is merged through a reviewed pull request. All repositories follow git-flow.

Hardware. No special hardware is needed. We work on our own computers; the client offers the institute's PC pool and servers if needed.

Testing. Every public function of kgg-core and every REST endpoint has automated tests, which run on every pull request. A pull request is merged only after another team member has reviewed it and all tests pass. At the end of each two-week iteration, the client accepts finished requirements in a demo against their acceptance criteria. The client provides freely available sample data as test data.

Target environment. In the long term, the KGG runs on the institute's server as part of its Django-based backend, which calls it over the REST API. During the project, it runs locally with a local database. If needed, the client provides a Dockerfile or a virtual machine.

Artefacts. We deliver the source code of all three repositories with their automated tests. Every requirement comes with tests for its new behaviour and updated user documentation, and each repository has a README on setup and usage. Releases are tagged on the master branch. kgg-core also holds this specification, all meeting minutes and our process documentation.

Form of delivery. All artefacts are in our public GitHub organisation. At the end, we push all repositories to the institute's GitLab, as agreed with the client.

Change requests. Change requests come via the iteration meeting or Discord. We record each one as a GitHub issue with acceptance criteria and estimate it. The client sets its priority, and we plan it into a coming iteration. The minimum scope only changes by mutual agreement, recorded in the minutes.

Minimum scope. The client named storing, reading, editing, updating and versioning triples as the main focus. The project is passed when the client's backend can send triples over the REST API, the KGG stores them in a local database, they can be read, edited and updated over the API, and every change is recorded as a version with Linked Data Event Streams (LDES). A reviewer can also accept or reject triples marked as unclear in the web frontend. A graph visualisation and Neo4j compatibility are not required but desirable.

Use of AI. The client allows us to generate code with AI if it is reviewed and the team understands it and can answer any question about it.

3. Risks

Team member is unavailable for a long time or leaves the team

a. Probability: Low. All team members are committed, and none of them intends to leave. Two members study abroad from February 2027 but keep working remotely, as registered with the course.

b. Impact: The loss of a team member would slow down the project. The most critical case would be the loss of the technically strongest member, since the others would have to acquire this member's knowledge.

c. Mitigation: Clear documentation, meeting minutes and clearly assigned tasks enable the remaining members to familiarise themselves with the work quickly and to redistribute the tasks. In the case of a long-term or permanent absence, we inform our course supervisor and notify the course organisers.

Team member (repeatedly) does not keep agreements

a. Probability: Medium.

b. Impact: Other team members would have to take over the affected tasks, or milestones would be missed. This would delay the project and lead to point deductions.

c. Mitigation: If a team member does not respond on Discord, we contact them via WhatsApp and, if necessary, by phone. If there is no response within one to two days, their task is reassigned and the course supervisor is informed. If a member fails to meet an agreed deadline, a member with more available time takes over, and the member concerned compensates for this later with additional work. If this happens repeatedly, the team reserves the right to ask the course organisers, with justification and evidence, to adjust the distribution of points.

Resources and their availability

a. Probability: Low. The required resources include a virtual machine, sample data in RDF format and the ontologies.

b. Impact: If one of these resources is not provided, the results would be less accurate and might not fully meet the client's expectations.

c. Mitigation: If the sample data in RDF format or the ontologies are not provided, we work with small example triples and a publicly available ontology until the client's data arrives. Without a virtual machine, we can still continue our work.

Client is unavailable or does not keep agreements

a. Probability: Medium. The client has already supervised two TPSE teams, but delays in responses or deliveries can still occur.

b. Impact: The impact would be severe. The project would slow down or come to a standstill, milestones would be missed, and our work might go in the wrong direction.

c. Mitigation: We address problems with the client as early and openly as possible. If the client changes agreements afterwards, we clarify this with the course supervisor on the basis of our documentation and meeting minutes, which show what was agreed. If information is missing, we continue working with assumptions based on our best knowledge. Once the client responds, we discuss these assumptions with them. If Ning Xia cannot be reached before the next meeting and does not attend it either, we inform the course supervisor and subsequently the course organisers.

4. Legal

Software Licence

We propose to publish the software as open source under the MIT License. The final licence will be agreed with the client. This means that the entire source code is publicly accessible and may be modified without restriction. The software may be used free of charge for any purpose, including integration into commercial products. The developers assume no liability for damages, errors, or data loss resulting from the use of the software. The original copyright notice of the authors must be retained in all copies or substantial portions of the software.

Data Confidentiality and Non-Disclosure Agreements

All data provided to the team within the scope of this project are publicly available and are not subject to any confidentiality restrictions. Consequently, no non-disclosure agreement (NDA) applies to this project.

Use by the Team

As the software is open source, team members may continue to use it and refer to it after the project. We will confirm this with the client.

Communication

Discord serves as the primary communication channel between the client (Chair of Fluid Systems) and the development team. Synchronisation meetings between the client and the development team take place every two weeks to discuss current progress, emerging technical challenges, and open questions.

Primary contact: Surkhabjit Singh (surkhabjit.singh@stud.tu-darmstadt.de)

Response time: We answer enquiries sent to the primary contact or posted on Discord within one business day where possible, and at the latest within two business days. In return, we ask the client to respond to our enquiries within three business days.